DUOC UC — ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Desarrollo Fullstack II (DSY1104) — Evaluación Parcial 2

Pastelería Mil Sabores

Informe de Pruebas Unitarias y Cobertura de Código

Versión 2.0 (Testing Automatizado con Vitest)

Documento:	Informe Técnico de Testing y Cobertura de Código
Proyecto:	Pastelería Mil Sabores (Frontend SPA)
Asignatura:	Desarrollo Fullstack II (DSY1104)
Institución:	Duoc UC
Integrantes:	*[Nombre del Alumno 1 / Nombre del Alumno 2]*
Docente:	*[Nombre del Profesor / Docente]*
Fecha de Ejecución:	17 de Octubre de 2026
Estado de Pruebas:	**171 Pasadas / 0 Fallidas (100% de Éxito)**

1. Introducción

En el ciclo de vida del software profesional, el aseguramiento de la calidad (QA) y la automatización de pruebas unitarias son disciplinas indispensables para garantizar que cada componente, módulo y función opere de acuerdo con los requisitos técnicos sin regresiones. Para el proyecto **Pastelería Mil Sabores** en su Evaluación Parcial 2, se estructuró una suite de pruebas automatizadas mediante el framework Vitest.

El presente informe constituye un documento técnico formal, independiente y complementario a la Especificación de Requisitos de Software (ERS V2). Mientras que el ERS formaliza los requerimientos funcionales y de arquitectura, este informe audita y certifica la verificación del código, documentando la estrategia de aislamiento, el detalle de los casos de prueba ejecutados y las métricas objetivas de cobertura provistas por el motor nativo `@vitest/coverage-v8`.

2. Objetivos del Proceso de Pruebas

2.1 Objetivo General

Evaluar, certificar y documentar la calidad funcional, estabilidad y cobertura de código del frontend de Pastelería Mil Sabores mediante pruebas unitarias automatizadas con Vitest, alcanzando un umbral de cobertura superior al 80% exigido por la rúbrica institucional de la asignatura Desarrollo Fullstack II en Duoc UC.

2.2 Objetivos Específicos

- Verificar el renderizado correcto y libre de errores de los componentes de la interfaz de usuario bajo Atomic Design (átomos, moléculas, organismos, plantillas y páginas).
- Validar interacciones de usuario críticas: agregar productos al carrito, modificar cantidades, filtrar por categoría gastronómica, buscar por texto, autenticación y envío de formularios.
- Comprobar la lógica de negocio y cálculos de precios efectivos con descuento mediante la función `getPrecioEfectivo()` y la correcta formulación de pedidos con identificador único `PED-timestamp-random`.
- Garantizar la persistencia y migración de datos en LocalStorage (claves `milSabores_*`) simuladas en entorno controlado `jsdom` con aislamiento estricto entre pruebas.
- Cuantificar y analizar el porcentaje de cobertura de líneas, ramas, funciones y sentencias, identificando áreas con cobertura completa y oportunidades de optimización.

3. Entorno de Ejecución y Herramientas

La infraestructura de pruebas automatizadas utiliza herramientas modernas estándar del ecosistema Vite y React 19. Todas las versiones corresponden exactamente a las especificadas en `package.json`:

Herramienta / Dependencia	Versión Real	Rol y Propósito en la Suite de Pruebas
Vitest	5.0.5	Test runner nativo impulsado por Vite para ejecución de pruebas unitarias ultrarrápidas y aserciones.
Vite	6.3.1	Empaquetador y motor de transformación de ES Modules y JSX para compilación en caliente.
React Testing Library	16.3.3	Utilidades centradas en el usuario (`render` , `screen` , `fireEvent`) para probar componentes de React.
@testing-library/jest-dom	7.0.1	Matchers semánticos para aserciones del DOM (`toBeInTheDocument` , `toHaveTextContent` , etc.).
@testing-library/user-event	14.6.8	Simulación fidedigna de interacciones de usuario (clicks, tecleo, cambios de input).
jsdom	30.1.3	Emulación completa del DOM de navegador en Node.js, incluyendo Window, Document y Web Storage API.
@vitest/coverage-v8	5.0.5	Motor de instrumentación nativo de V8 para medir cobertura de líneas, ramas, funciones y sentencias.

4. Estrategia de Pruebas

La estrategia se fundamenta en pruebas unitarias de caja blanca y pruebas de comportamiento funcional orientadas al usuario (Behavior-Driven Testing). Las pruebas se clasifican en las siguientes categorías:

- **Pruebas de Renderizado y Estructura Visual:** Verifican que los componentes atómicos (`Alert` , `Badge` , `Button` , `Input`), moléculas (`FormField` , `QuantityControl` , `SearchBar`) y organismos (`Navbar` , `Footer` , `ProductCard` , `CartItem`) se monten en el DOM con las clases de Bootstrap, atributos semánticos de accesibilidad (`role` , `aria-label`) y textos requeridos.
- **Pruebas de Interacción y Eventos:** Mediante `fireEvent` y `userEvent` se simulan clicks de botones ("Añadir al carrito", "Eliminar", "Limpiar búsqueda"), digitación en campos de texto (búsqueda, login, registro, checkout) y selección de categorías.
- **Pruebas de Validación y Manejo de Errores:** Se verifican escenarios donde el usuario introduce datos inválidos (contraseñas cortas, correos mal estructurados, campos vacíos en checkout, precios no enteros en administración), comprobando la emisión de alertas contextuales de error sin romper la aplicación.
- **Pruebas de Lógica de Negocio Pura (Servicios):** Pruebas aisladas sobre las funciones de los 5 servicios (`productosService` , `carritoService` , `pedidosService` , `usuariosService` y `blogService`), verificando reglas de cálculo de ofertas (`getPrecioEfectivo`), filtrados, generación de identificadores `PED-` y subtotales/totales.
- **Pruebas de Persistencia y Migración:** Verificación de lectura, escritura y sincronización en LocalStorage con simulación de versiones de catálogo (`CATALOG_VERSION = 3`).

> **Nota Académica de Alcance:** No se realizaron pruebas de carga ni pruebas de penetración o ciberseguridad avanzada, ya que el sistema opera en esta etapa puramente en el cliente y tales pruebas exceden los requisitos de la Evaluación Parcial 2.

5. Configuración de Vitest y Ejecución

5.1 Configuración en vite.config.js

La suite de testing se encuentra integrada en `vite.config.js` con los siguientes parámetros clave:

```
test: {
  environment: 'jsdom',
  globals: false,
  setupFiles: ['./test/setup.js'],
  include: ['test/**/*.{test,spec}.{js,jsx}'],
  coverage: {
    provider: 'v8',
    reporter: ['text', 'html'],
    include: ['src/**/*.{js,jsx}'],
    exclude: ['src/main.jsx', 'src/assets/**'],
  },
}
```

5.2 Comandos Oficiales de Ejecución

- `npm.cmd run test:coverage`: Ejecuta todas las pruebas en lote y genera las tablas de cobertura por consola y el reporte interactivo en `coverage/index.html`.
- `npm.cmd run test:run`: Ejecuta todas las pruebas unitarias una sola vez sin modo watcher.
- `npm.cmd test`: Modo interactivo watcher para desarrollo activo.

6. Casos de Prueba Ejecutados

A continuación se detallan los casos de prueba ejecutados en los 25 archivos de especificación (`test/*.spec.jsx`). Los 171 casos obtuvieron estado **PASS**:

6.1 Catálogo, Búsqueda, Detalle y Ofertas

ID Caso	Archivo de Prueba	Componente / Módulo	Comportamiento Evaluado	Resultado
CP-CAT-01	test/Productos.spec.jsx	Productos.jsx	Renderiza el catálogo y los productos iniciales.	PASS
CP-CAT-02	test/Productos.spec.jsx	Productos.jsx	Filtra productos en tiempo real según término de búsqueda.	PASS
CP-CAT-03	test/Productos.spec.jsx	Productos.jsx	Valida que las 28 imágenes apunten a /images/productos/.	PASS
CP-DET-01	test/DetalleProducto.spec.jsx	DetalleProducto.jsx	Muestra detalles, precio, ingredientes e imagen del producto.	PASS
CP-DET-02	test/DetalleProducto.spec.jsx	DetalleProducto.jsx	Muestra mensaje amigable cuando el ID del producto no existe.	PASS
CP-OFE-01	test/Ofertas.spec.jsx	Ofertas.jsx	Filtra y muestra exclusivamente productos en oferta.	PASS
CP-SRV-01	test/productosService.spec.jsx	productosService.js	Calcula precio efectivo con getPrecioEfectivo() correctamente.	PASS

6.2 Carrito, Checkout y Compras

ID Caso	Archivo de Prueba	Componente / Módulo	Comportamiento Evaluado	Resultado
CP-CAR-01	test/Carrito.spec.jsx	Carrito.jsx	Renderiza estado vacío y botón para explorar productos.	**PASS**
CP-CAR-02	test/Carrito.spec.jsx	Carrito.jsx	Permite incrementar, decrementar y eliminar ítems del carrito.	**PASS**
CP-CHK-01	test/Checkout.spec.jsx	Checkout.jsx	Muestra advertencia cuando el carrito está vacío e impide compra.	**PASS**
CP-CHK-02	test/Checkout.spec.jsx	Checkout.jsx	Valida campos obligatorios: email, teléfono, dirección, comuna.	**PASS**
CP-EXIT-01	test/CompraExitosa.spec.jsx	CompraExitosa.jsx	Muestra confirmación de orden con ID PED- y datos de despacho.	**PASS**
CP-FALL-01	test/CompraFallida.spec.jsx	CompraFallida.jsx	Muestra mensaje de error y enlace para reintentar desde carrito.	**PASS**

6.3 Autenticación, Administración y Rutas Protegidas

ID Caso	Archivo de Prueba	Componente / Módulo	Comportamiento Evaluado	Resultado
CP-LOG-01	test/Login.spec.jsx	Login.jsx	Inicia sesión para cliente y redirige a la página principal.	PASS
CP-LOG-02	test/Login.spec.jsx	Login.jsx	Inicia sesión para administrador y redirige a /admin.	PASS
CP-REG-01	test/Registro.spec.jsx	Registro.jsx	Registra nueva cuenta y redirige a login tras éxito.	PASS
CP-PROT-01	test/RutaProtegida.spec.jsx	RutaProtegida.jsx	Redirige a /login a usuarios sin sesión activa.	PASS
CP-PROT-02	test/RutaProtegida.spec.jsx	RutaProtegida.jsx	Redirige a / a usuarios con rol cliente que intenten /admin.	PASS
CP-ADM-01	test/Admin.spec.jsx	Admin.jsx	Renderiza panel de control y tabla de productos.	PASS
CP-ADM-02	test/Admin.spec.jsx	Admin.jsx	Permite dar de alta un producto y actualizar inventario.	PASS
CP-ADM-03	test/Admin.spec.jsx	Admin.jsx	Permite entrar en modo edición y guardar cambios.	PASS
CP-ADM-04	test/Admin.spec.jsx	Admin.jsx	Permite eliminar producto tras confirmación de usuario.	PASS

6.4 Vistas Institucionales y Atomic Design

ID Caso	Archivo de Prueba	Componente / Módulo	Comportamiento Evaluado	Resultado
CP-HOM-01	test/Home.spec.jsx	Home.jsx	Renderiza banner héroe, productos destacados y enlaces.	PASS
CP-NOS-01	test/Nosotros.spec.jsx	Nosotros.jsx	Renderiza historia, valores y fotografías de cocina.	PASS
CP-BLG-01	test/Blog.spec.jsx	Blog.jsx	Renderiza lista de artículos y categorías de blog.	PASS
CP-CON-01	test/Contacto.spec.jsx	Contacto.jsx	Valida y simula el envío del formulario de contacto.	PASS
CP-ATM-01	test/atomicComponents.spec.jsx	Alert, Badge, Button, Input	Verifica props, clases y comportamiento de átomos.	PASS

7. Uso de Mocks y Estrategia de Aislamiento

Para asegurar que las pruebas sean deterministas, independientes y reproducibles, la suite aplica un riguroso protocolo de aislamiento:

- **Limpieza de LocalStorage:** En cada archivo de prueba que interactúa con la persistencia, se invoca `beforeEach(() => localStorage.clear())` para garantizar que ningún dato residual de una prueba afecte a la siguiente.
- **Limpieza del DOM Virtual:** En `test/setup.js` se ejecuta `afterEach(() => cleanup())`, desmontando todos los árboles de componentes creados por React Testing Library y liberando memoria.
- **Mocks de APIs del Navegador:** Se implementan mocks controlados para `window.confirm` (simulando confirmación o cancelación del usuario en la eliminación de productos), `window.scrollTo` (evitando errores por funciones no implementadas en jsdom) y `console.error`.
- **Restauración de Espías:** Mediante `vi.restoreAllMocks()` se restablece el comportamiento original de funciones espiadas tras cada test suite.

8. Resultados de Ejecución Oficial

La ejecución oficial auditada arrojó los siguientes resultados globales:

> **Resultado de la Ejecución:** 25 archivos de prueba ejecutados y aprobados al 100%. Total de 171 pruebas pasadas con 0 fallos y 0 pruebas omitidas.

Parámetro de Ejecución	Valor Auditado
Fecha y Hora de Ejecución:	17 de Octubre de 2026, 16:38:36 (GMT-3)
Total de Archivos de Prueba:	25 evaluados / 25 aprobados (100%)
Total de Pruebas Unitarias:	**171 ejecutadas / 171 aprobadas (0 fallidas, 0 omitidas)**
Duración Total:	24.86 segundos
Tasa de Éxito:	**100.00%**

9. Análisis de Cobertura de Código

9.1 Métricas Globales Obtenidas

A través del proveedor `@vitest/coverage-v8`, se instrumentó todo el código fuente ubicado en `src/`. A continuación se comparan los resultados reales con el umbral mínimo institucional del 80%:

Métrica de Cobertura	Resultado Real	Umbral Mínimo (Rúbrica)	Estado y Desviación
Líneas (% Lines)	**89.54%**	80.00%	**Aprobado (+9.54%)**
Ramas (% Branch)	**90.35%**	80.00%	**Aprobado (+10.35%)**
Funciones (% Funcs)	**89.23%**	80.00%	**Aprobado (+9.23%)**
Sentencias (% Stmts)	**88.95%**	80.00%	**Aprobado (+8.95%)**

9.2 Desglose de Cobertura por Capas y Módulos

Capa / Módulo	% Líneas	% Ramas	% Funciones	% Sentencias
src/ (App.jsx)	100.00%	100.00%	100.00%	100.00%
src/components/atoms	100.00%	100.00%	100.00%	100.00%
src/components/molecules	100.00%	91.11%	100.00%	100.00%
src/components/organisms	88.57%	100.00%	87.50%	88.57%
src/components/templates	100.00%	100.00%	100.00%	100.00%
src/pages	87.86%	88.20%	85.45%	87.78%
src/services	91.02%	87.67%	94.73%	89.53%

9.3 Análisis de Archivos con Mayor y Menor Cobertura

- **Archivos con Cobertura Plena (100% Líneas):** `App.jsx`, `Alert.jsx`, `Badge.jsx`, `Button.jsx`, `Input.jsx`, `FormField.jsx`, `QuantityControl.jsx`, `SearchBar.jsx`, `CartItem.jsx`, `CartSummary.jsx`, `Footer.jsx`, `Navbar.jsx`, `RutaProtegida.jsx`, `AuthLayout.jsx`, `MainLayout.jsx`, `PageHeader.jsx`, `Login.jsx`, `Registro.jsx`, `CompraFallida.jsx`, `CompraExitosa.jsx`, `blogService.js` y `carritoService.js`.
- **Archivos con Menor Cobertura:** `Home.jsx` (33.33% líneas), `Nosotros.jsx` (33.33% líneas), `Blog.jsx` (66.66% líneas), `DetalleBlog.jsx` (71.42% líneas), `Checkout.jsx` (79.48% líneas) y `DetalleProducto.jsx` (81.25% líneas).
- **Justificación Técnica:** En Home y Nosotros, las pruebas validan títulos, secciones clave y presencia visual, pero no disparan deliberadamente fallbacks de imágenes secundarias. En Checkout, las líneas no cubiertas corresponden a ramas secundarias de validación en caso de manipulación extrema de campos no contemplados en el flujo estándar.

10. Hallazgos y Oportunidades de Mejora

10.1 Hallazgos Detectados Durante la Auditoría

1. **Excepciones de Almacenamiento Saturado:** En los servicios, los bloques `catch` en caso de que `localStorage` esté lleno o deshabilitado no son forzados en las pruebas unitarias comunes, lo que deja

algunas líneas de log *uncovered*.

2. **Migración Legacy de Catálogo:** En `productosService.js`, las líneas 317-337 (fusión con versiones anteriores al catálogo 3) no se ejecutan cuando el entorno de test inicia directamente en versión 3.
3. **Advertencias de Consola por React Router:** Se detectaron advertencias informativas por rutas simuladas en pruebas individuales sin impacto en el funcionamiento.

10.2 Recomendaciones Técnicas para la Evaluación Parcial 3

1. Inyectar pruebas específicas con mocks que simulen fallo forzado de `localStorage.setItem` (`QuotaExceededError`) para cubrir al 100% los bloques `try/catch` de los servicios.
2. Implementar pruebas de integración End-to-End (E2E) con Playwright o Cypress para validar flujos continuos (desde la elección en catálogo hasta la confirmación de checkout) en un navegador real.

11. Conclusiones

La ejecución automatizada de pruebas unitarias sobre **Pastelería Mil Sabores** certifica un alto estándar de calidad, fiabilidad y estabilidad en su versión 2. Con un **100% de pruebas aprobadas (171 de 171)** distribuidas en 25 suites de especificación y una cobertura de líneas del **89.54%** (excediendo ampliamente la exigencia institucional del 80%), el sistema garantiza que los componentes visuales de Atomic Design, la navegación SPA, la gestión de inventario y la lógica de negocio operan de manera correcta y consistente.

12. Anexos y Referencias

- **Comando Oficial de Reproducción:** `npm.cmd run test:coverage`
- **Directorio de Archivos de Prueba:** `/test/*.spec.jsx` (25 archivos)
- **Archivo de Configuración de Setup:** `/test/setup.js`
- **Informe HTML Detallado Interactivo:** `/coverage/index.html` (disponible localmente en la carpeta `coverage/` para navegación archivo por archivo con resaltado de sintaxis de líneas cubiertas).